A Report on

Breakout game

Submitted By

Shubham Tanna E026

Viha Jain E027

Virati Chordiya E017

Kashvi Bhagat E034

Shri Vile Parle Kelvani Mandal's

Mukesh Patel School of Technology & Management Engineering

Department of Basic Sciences and Humanities

Vile Parle (w), Mumbai- 400056

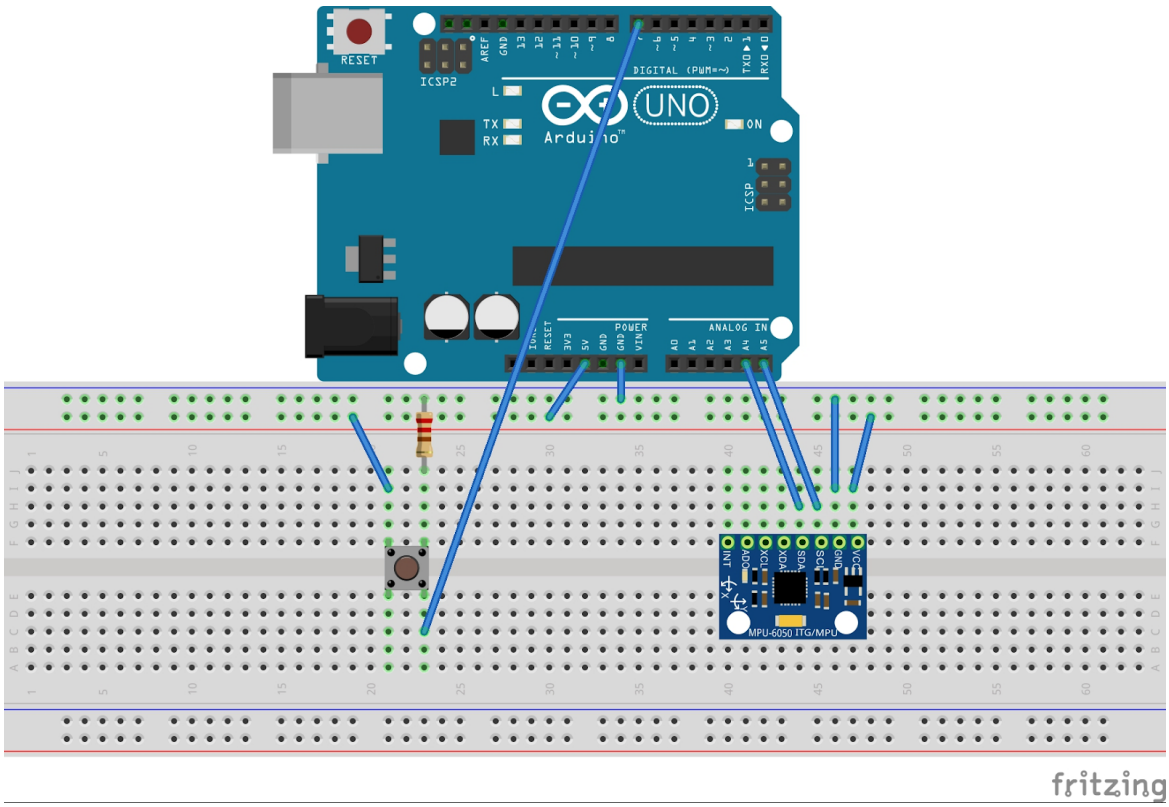
Chapter 1 Introduction (Brief introduction about the project.)

The project is a recreational game called breakout.

Chapter 2 Components (List of components and description)

1. The components used are:
2. Arduino Uno
3. Gyroscope GY-512 Module
4. Push Button
5. 10k ohm resistor
6. Jumper wires
7. Breadboard

Chapter 3 Working principle (Circuit diagram if any)



The SCL pin in the gyroscope is connected to the A5 (Analog 5) pin in the Arduino Uno.

The SDA pin of the gyroscope is connected to A4 (Analog 4) in the Arduino Uno.

The Button is connected to the digital pin 7 on the Uno board.

The 5V supply and Ground from the Arduino are connected to the vertical connections of the breadboard.

The push button and the sensor's power supply are connected to the 5V.

The 10k ohm resistor is connected to the button on the ground wire.

Chapter 4 Software description (Along with working code)

The Arduino software code is used to retrieve data from the gyroscope GY 512 using pre-existing libraries and also set up a serial communication between the arduino and the computer that the arduino is attached to.

The Processing software code is used to play the game and interface using the data fed into the computer by the arduino serial communication.

Arduino Code:

```
#include <Bounce2.h>
#include <Wire.h>

const unsigned int Button_Pin=7;
const int MPU=0x68;
int16_t AcX,AcY,AcZ;
const int NUM_AXES=3;

const int BUFFER_SIZE=16;
int buffer[NUM_AXES][BUFFER_SIZE];
int buffer_pos[NUM_AXES]={0};
```

```
Bounce button(Button_Pin, 20);

void setup(){
  Wire.begin();
  Wire.beginTransmission(MPU);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);
  Serial.begin(19200);
  pinMode(Button_Pin, INPUT);
}

int get_axis(const int axis){
  delay(1);
  buffer[axis][buffer_pos[axis]]=Wire.read()<<8|Wire.read();
  buffer_pos[axis]=(buffer_pos[axis]+1)%BUFFER_SIZE;
  long sum=0;
  for(int i=0; i<BUFFER_SIZE; i++)
  sum+= buffer[axis][i];
  return round(sum/BUFFER_SIZE);
  }

  int get_x(){return get_axis(0);}
  int get_y(){return get_axis(1);}
  int get_z(){return get_axis(2);}
```

```
void loop(){
  Wire.beginTransmission(MPU);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU,6,true);
  Serial.print(get_x());
  Serial.print(" ");
  Serial.print(get_y());
  Serial.print(" ");
  Serial.print(get_z());
  Serial.print(" ");
  if(button.update())
    Serial.println(button.read()==HIGH ? "1" : "0");
  else
    Serial.println("0");

  delay(50);
}
```

Processing code:

```
import processing.serial.*;

Serial arduinoPort;

final int COLUMNS=7;
final int ROWS=4;
```

```
final int BALL_RADIUS=8;
final int BALL_DIAMETER=BALL_RADIUS*2;
final int MAX_VELOCITY=8;
final int MARGIN=10;
final int PADDLE_WIDTH=60;
final int PADDLE_HEIGHT=15;
final int BRICK_WIDTH=40;
final int BRICK_HEIGHT=20;
final int HEIGHT=300;
final int LINE_FEED=10;

int px, py;
int vx, vy;
int xpos = 150;
int[][] bricks= new int[COLUMNS][ROWS];

boolean buttonPressed=false;
boolean paused=true;
boolean done=true;

void setup(){
  size(300, 300);
  noCursor();
  textFont(loadFont("Verdana-Bold-36.vlw"));
  initGame();
  String portName = Serial.list()[2];
```

```
  //println(Arduino.list());
  arduinoPort=new Serial(this, portName, 19200);
  arduinoPort.bufferUntil(10);
}

void initGame(){
  initBricks();
  initBall();
}

void initBricks(){
  for(int x=0; x<COLUMNS; x++)
  for(int y=0; y<ROWS; y++)
  bricks[x][y]=1;
}

void initBall(){
  px=width/2;
  py=height/2;
  vx=int(random(-MAX_VELOCITY, MAX_VELOCITY));
  vy=-2;
}

void draw(){
  background(0);
  stroke(255);
```

```
  strokeWeight(3);

  fill(#FFC0CB);
  textSize(10);
  text("Made by Shubham", 150, 10);

  done=drawBricks();
  if(done){
    paused=true;
    printWinMessage();
  }
  if(paused)
    printPauseMessage();
  else
    updateGame();

  drawBall();
  drawPaddle();
}

boolean drawBricks(){
  boolean allEmpty=true;
  for(int x=0; x<COLUMNS; x++){
    for(int y=0; y<ROWS; y++){
      if(bricks[x][y]>0){
        allEmpty=false;
```

```
        fill(0, 0, 100+y*8);
        rect(
          MARGIN+x*BRICK_WIDTH,
          MARGIN+y*BRICK_HEIGHT,
          BRICK_WIDTH,
          BRICK_HEIGHT
        );
      }
    }
  }
  return allEmpty;
}

void drawBall(){
  strokeWeight(1);
  fill(128, 0, 0);
  ellipse(px, py, BALL_DIAMETER,  BALL_DIAMETER);
}

void drawPaddle(){
  int x= xpos - PADDLE_WIDTH/2;
  int y= height - 25;
  strokeWeight(1);
  fill(128);
  rect(x, y, 60, 15);
}
```

```
void printWinMessage(){
  fill(225);
  textSize(36);
  textAlign(CENTER);
  text("YOU WIN", width/2, height*2/3);
}

void printPauseMessage(){
  fill(128);
  textSize(16);
  textAlign(CENTER);
  text("Press button to continue", width/2, height*5/6);
}

void updateGame(){
  if(ballDropped()){
    initBall();
    paused=true;
  } else{
    checkBrickCollision();
    checkWallCollision();
    checkPaddleCollision();
    px+=vx;
    py+=vy;
  }
```

```
}

boolean ballDropped(){
  return py+vy> height - BALL_RADIUS;
}

boolean inXRange(final int row, final int v){
  return px + v > row*BRICK_WIDTH&&
        px+v < (row+1)*BRICK_WIDTH+BALL_DIAMETER;
}

boolean inYRange(final int col, final int v){
  return py+v> col*BRICK_HEIGHT&&
        py+v< (col+1)*BRICK_HEIGHT+BALL_DIAMETER;
}

void checkBrickCollision(){
  for(int x=0; x<COLUMNS; x++){
    for(int y=0; y<ROWS; y++){
      if(bricks[x][y]>0){
        if(inXRange(x, vx)&& inYRange(y, vy)){
          bricks[x][y]=0;
          if(inXRange(x, 0))
          vy=-vy;
          if(inYRange(y, 0))
          vx=-vx;
```

```
      }
     }
    }
  }
}

void checkWallCollision(){
  if(px+vx < BALL_RADIUS || px+vx > width - BALL_RADIUS)
  vx=-vx;

  if(py+vy < BALL_RADIUS || py+vy > height - BALL_RADIUS)
  vy=-vy;
}

void checkPaddleCollision(){
  final int cx=xpos;
  if(py+vy >=height - (PADDLE_HEIGHT + MARGIN + 6)&&
    px >= cx - PADDLE_WIDTH/2 &&
    px <= cx + PADDLE_WIDTH/2)
    {
     vy=-vy;
     vx=int(
       map(
         px - cx,
         -(PADDLE_WIDTH/2), PADDLE_WIDTH/2,
         -MAX_VELOCITY,
```

```
        MAX_VELOCITY
      )
    );
  }
}

void serialEvent(Serial port){
 final String arduinoData=port.readStringUntil(LINE_FEED);
 if(arduinoData!=null){
  println(arduinoData);
  final int[] data=int(split(trim(arduinoData), ' '));
   if(data.length==4){
    buttonPressed=(data[3]==1);
   if(buttonPressed){
      paused= !paused;
      if(done){
        done=false;
        initGame();
      }
    }

    if(!paused)
    xpos= int(map(data[0]-1500, 252, 0, 0, 150));
  }
 }
}
```

Chapter 5 Advantages, limitations and applications

Advantages: Leisure, entertainment etc

Limitations: Wired control, gyroscope being too sensitive making control difficult

Applications: Entertainment systems, gaming consoles etc.

Chapter 6 Conclusion and future work

Adding “Lives” to the game, if the ball bounces out 3 times, you lose.

Chapter 7 References

Mert Arduino:
https://www.instructables.com/Arduino-MPU6050-GY521-6-Axis-Accelerometer-Gyro-3D/

Processing:

https://processing.org/tutorials/

https://www.toptal.com/game/ultimate-guide-to-processing-simple-game